FoodBridge — React Frontend Mentorship

Day 2 Notes — Login/Register Pages and Controlled Forms

0. Approach to Styling Going Forward

Decided not to learn raw CSS in depth. Tailwind's utility classes are used directly, and each new class is explained in plain language the first time it appears, instead of studying CSS theory (box model, selectors, specificity, etc.) separately. Recommended installing the 'Tailwind CSS IntelliSense' VS Code extension for live class previews.

1. Login Page (Static UI First)

Built the Login form UI first, without any logic, at src/pages/Login/Login.jsx.

```
<div className="flex justify-center items-center min-h-screen bg-gray-50">
  <div className="bg-white p-8 rounded-lg shadow-md w-full max-w-sm">
    ...form with Email + Password fields + Login button...
  </div>
</div>
```

New Tailwind classes learned:

- min-h-screen -> element takes at least full screen height
- justify-center / items-center -> centers content horizontally / vertically in a flex container
- rounded / rounded-lg -> rounded corners (lg = bigger radius)
- shadow-md -> medium drop shadow, gives a card-like look
- w-full max-w-sm -> full width but capped at a small max width
- mb-4 / mb-6 -> margin-bottom (spacing below element)
- border border-gray-300 -> thin gray border
- px-3 py-2 -> horizontal / vertical padding inside an element

Route added in App.jsx:

```
import Login from './pages/Login/Login'
...
<Route path="/login" element={<Login />} />
```

2. Register Page

Same layout pattern as Login, with extra fields: Full Name, and a Role dropdown (select) matching backend roles.

```
<select className="w-full border border-gray-300 rounded px-3 py-2">
  <option value="">Select role</option>
  <option value="DONOR">Donor</option>
  <option value="NGO">NGO</option>
  <option value="VOLUNTEER">Volunteer</option>
</select>
```

ADMIN is intentionally left out of the self-registration dropdown — admins are created directly in the database or by another admin, not via public signup.

Route added in App.jsx:

```
import Register from './pages/Register/Register'
...
<Route path="/register" element={<Register />} />
```

3. Making the Login Form Interactive with useState

Static forms don't remember what the user types. useState lets a component hold and update its own data (state), and React automatically re-renders when it changes.

```
import { useState } from 'react'

function Login() {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')

  const handleSubmit = (e) => {
    e.preventDefault()
    console.log('Email:', email)
    console.log('Password:', password)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  )
}
```

Key concepts explained:

- useState('') -> creates a state variable (email) + updater function (setEmail); updating state re-renders the component
- value={email} on the input -> makes it a 'controlled component' — the input's displayed value always comes from React state, not the browser's own memory
- onChange={(e) => setEmail(e.target.value)} -> fires on every keystroke; e.target.value is the current text in the box
- onSubmit={handleSubmit} + e.preventDefault() -> stops the browser's default full-page-reload form submission, so React/JS handles it instead (this is where the login API call will go next)

Verified by typing into Email/Password fields and checking the browser console (F12) for the logged values on submit.

4. Git Commit (End of Day 2)

```
git add .
```

```
git commit -m "feat: build Login/Register UI and add controlled form state to Login

- Created Login page with email/password form (Tailwind styled card layout)
- Created Register page with name/email/password/role fields
- Added /login and /register routes in App.jsx
- Implemented useState in Login for controlled inputs (email, password)
- Added handleSubmit with e.preventDefault() and console logging (API call pending)"
```

5. Next Steps (Day 3)

- Add useState to Register page (practice — same pattern as Login)
- Set up Axios instance (baseURL, headers) in src/api/
- Create AuthContext for storing JWT + logged-in user + role
- Connect Login form to backend login API and store JWT on success
- Add toast notifications (react-toastify) for success/error feedback